



KOMBINASI STRUKTUR ALGORITMA PEMROGRAMAN





TABLE OF CONTENT

Pendahuluan

01

Mengenali Alur Percabangan
dalam Perulangan

02

Definisi Arsip Beruntun

03

Pendeklarasian Arsip di dalam
Algoritma

04

Contoh implementasi Arsip
Beruntun

05

Pembacaan Arsip Beruntun

06

Mengenali Alur Perulangan dalam
Percabangan

07

Proses Rekursif

08

Definisi Rekursif

09

Contoh Implementasi Proses
Rekursif

10



PENDAHULUAN



Bab ini akan membahas terkait alasan mengapa perlu mempelajari dan memahami bagaimana kombinasi struktur algoritma pemrograman berperan penting dalam penguasaan keilmuan pemrograman dasar yang nantinya dapat digunakan untuk mempermudah pekerjaan terkait dengan pemrograman.



PENDAHULUAN

Pemaparan dari bab ini bertujuan untuk memberikan pemahaman lebih lanjut mengenai lanjutan dari bab matriks, percabangan dan perulangan yang telah kita pelajari sebelumnya.





MENGENALI ALUR PERCABANGAN DALAM PERULANGAN



Kombinasi struktur algoritma pemrograman meliputi bagaimana kita dapat mengenali alur percabangan dalam perulangan beserta implementasinya.



MENGENALI ALUR PERCABANGAN DALAM PERULANGAN

Pada struktur pemilihan atau percabangan, terdapat pemeriksaan kondisi/syarat yang harus dipenuhi, yang kemudian akan memilih perintah apa yang akan dilakukan jika syarat tersebut dipenuhi. Perintah tidak lagi dikerjakan secara beruntun seperti pada struktur runtunan, tetapi berdasarkan syarat yang harus dipenuhi (Uce Indahyanti et al, 2020).



MENGENALI ALUR PERCABANGAN DALAM PERULANGAN (2)



Definisi arsip (file) atau berkas adalah struktur penyimpanan data di dalam memori sekunder seperti disk. Data disimpan di dalam arsip agar sewaktu-waktu dapat dibuka kembali. Struktur arsip memungkinkan kita menyimpan data secara permanen dan mengaksesnya kembali bila perlu.

Pada umumnya, arsip menyimpan Informasi dari kategori yang sama. Misalnya, data karyawan disimpan di dalam arsip karyawan, data properti barang disimpan di dalam arsip properti, data keuangan disimpan di dalam arsip keuangan, dan sebagainya. Setiap arsip dikenali melalui namanya

(Rinaldi Munir, 2007).



MENGENALI ALUR PERCABANGAN DALAM PERULANGAN (3)

Setiap item data yang direkam di dalam arsip disebut rekaman (record). Semua rekaman di dalam arsip diorganisasikan penyimpanannya, dan pengaksesan rekaman di dalam arsip bergantung kepada metode pengorganisasiannya itu. Ada dua cara pengorganisasian data di dalam arsip: beruntun (sequential) dan acak (random) yang terakhir ini juga dikenal sebagai arsip akses langsung (direct access).



MENGENALI ALUR PERCABANGAN DALAM PERULANGAN (4)



Arsip beruntun (sequential file) menyimpan rekaman secara berurutan, rekaman yang satu sesudah rekaman yang lain. Untuk mengakses data tertentu, program harus mengakses mulai dari rekaman pertama sampai rekaman yang diinginkan. Pengaksesan arsip jenis ini pada umumnya sangat lambat, khususnya bila arsip berukuran besar. Namun, arsip beruntun mudah dibuat dan dipelihara (maintain).



MENGENALI ALUR PERCABANGAN DALAM PERULANGAN (5)

Pada arsip acak, rekaman diakses secara langsung, tanpa perlu memulai dari awal. Keuntungan jenis arsip ini, pengaksesan data individual lebih cepat daripada arsip beruntun, tetapi arsip acak lebih sulit dibuat dan dipelihara. Dari kedua cara pengorganisasian itu, pengorganisasian rekaman secara beruntun adalah metode yang paling sederhana.





DEFINISI ARSIP BERUNTUN

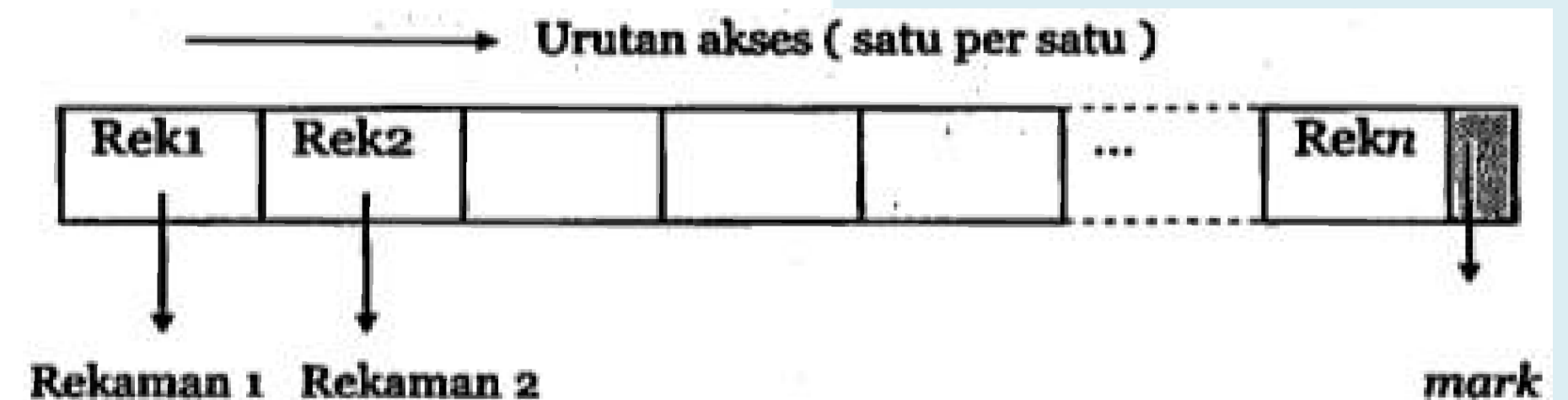


Arsip beruntun adalah sekumpulan rekaman bertipe sama yang diakses secara berurutan mulai dari rekaman pertama sampai rekaman yang dituju atau sampai dengan rekaman yang terakhir. Rekaman di dalam arsip beruntun diakses satu per satu, yaitu rekaman demi rekaman, secara searah.

DEFINISI ARSIP BERUNTUN



Gambar 1 memperlihatkan gambaran logik pengaksesan arsip beruntun. Setiap kotak pada gambar tersebut menyatakan sebuah rekaman. Kita dapat menganalogikan arsip beruntun seperti kumpulan lagu di dalam pita kaset. Untuk memainkan lagu pada urutan ke-6, kita harus memutar pita kaset mulai dari awal pita sampai awal lagu yang ke-6 ditemukan.





PENDEKLARASIAN ARSIP DI DALAM ALGORITMA



Sebelum melakukan pemrosesan arsip beruntun, arsip tersebut harus dideklarasikan terlebih dahulu. Cara mendeklarasikan arsip beruntun di dalam bagian DEKLARASI sebagai berikut (sebagai peubah):

DEKLARASI

arsip : File of tipe rekaman

Algoritma 1 Deklarasi arsip beruntun sebagai peubah (variabel)



PENDEKLARASIAN ARSIP DI DALAM ALGORITMA

Tipe rekaman dapat berupa tipe dasar (integer, real, char, boolean, string) atau tipe terstruktur (record). Setiap rekaman di dalam arsip beruntun harus bertipe sama, baik dari tipe dasar maupun bertipe terstruktur. Rekaman bertipe terstruktur terdiri atas satu atau lebih field yang bertipe tertentu.



PENDEKLARASIAN ARSIP DI DALAM ALGORITMA



Adapun cara mendeklarasikan arsip beruntun sebagai tipe bentukan sebagai berikut:

DEKLARASI

```
type nama tipe arsip : File of tipe rekaman { tipe bentukan }  
arsip : nama tipe arsip
```

Algoritma 2 Deklarasi arsip beruntun sebagai tipe bentukan



CONTOH IMPLEMENTASI ARSIP BERUNTUN



Arsip hanya dapat diproses jika sudah terdefinisi isinya. Langkah pertama dalam membuat arsip adalah menyiapkan arsip untuk perencanaan (menggunakan perintah Open dengan Kode = 2).



ARSIP BERUNTUN

Langkah kedua adalah membaca data yang akan direkam (misalnya pembacaan dari piranti masukan), barulah kemudian menuliskan data tersebut ke dalam arsip (perintah FWrite). Jika sebuah arsip sudah selesai diisi, arsip tersebut ditutup dengan perintah Close.



CONTOH IMPLEMENTASI ARSIP BERUNTUN (2)



Contoh 1. Mengisi arsip bilangan bulat dengan nilai 1 sampai n, Nilai n dibaca dari piranti masukan. Tanda akhir arsip didefinisikan 9999.

DEKLARASI

```
Bil : File of integer { arsip bilangan bulat }  
n : integer           { banyaknya bilangan bulat }  
i : integer           { pencacah pengulangan }
```

ALGORITMA:

```
Open(Bil, 2)      {buka arsip Bil untuk perekaman }  
read(n)  
for i ← 1 to n do  
    Fwrite(Bil, i)  
endfor  
Close(Bil)        { tutup arsip }
```

PEMBACAAN ARSIP BERUNTUN



Proses pembacaan arsip beruntun merupakan kebalikan dari proses pembuatan arsip. Langkah pertama dalam membaca arsip adalah menyiapkan arsip untuk pembacaan (menggunakan perintah Open dengan Kode = 1). Setiap kali akan membaca suatu rekaman di dalam arsip, kita harus memastikan bahwa tanda akhir arsip belum dicapai



PEMBACAAN ARSIP BERUNTUN

Rekaman dibaca satu per satu, dimulai dari rekaman pertama hingga rekaman yang diinginkan sudah ditemukan atau seluruh rekaman selesai dibaca. Karena kita tidak tahu jumlah data di dalam arsip, maka konstruksi pengulangan yang cocok adalah dengan WHILE seperti yang ditunjukkan pada Algoritma 13.



CONTOH IMPLEMENTASI ARSIP BERUNTUN (2)



PROGRAM PembacaanArsip

{ Skema pembacaan arsip beruntun tanpa penanganan kasus kosong }

DEKLARASI

type Rekaman : TipeRekaman

Arsip : File of Rekaman

Rek : Rekaman

ALGORITMA:

Inisialisasi

Open(Arsip, 1) { buka arsip untuk dibaca}

while not EOF(Arsip) do

 Fread (Arsip, Rek)

 Proses Rek

endwhile

{ EOF(Arsip) }

Terminasi

Close(Arsip)

MENGENALI ALUR PERULANGAN DALAM PERCABANGAN



Pada struktur perulangan, perulangan sendiri merupakan struktur dimana terdapat proses pengulangan perintah yang sama sebanyak n kali. Sebagai contoh untuk mencetak teks “Algoritma Pemrograman” sebanyak 100 kali pada layar monitor, hanya diperlukan beberapa baris perintah menggunakan teknik atau struktur perulangan tersebut. Tanpa harus menuliskan perintah yang sama sebanyak 100 kali (Uce Indahyanti et al, 2020).

MENGENALI ALUR PERULANGAN DALAM PERCABANGAN

Untuk mengenali alur perulangan dalam perabangan kita perlu mempelajari penerapannya melalui rekursifitas atau proses rekursif beserta turunan-turunannya. Selain itu, untuk mempelajari lebih dalam kita juga perlu memahami dalam penerapan atau implementasinya dalam studi kasus riil.



PROSES REKURSIF



Banyak objek di dalam matematika yang didefinisikan dengan cara menyatakan suatu proses (algoritma) yang menghasilkan objek tersebut. Sebagai contoh, π diperoleh dengan membagi keliling lingkaran (K) dengan diameternya (d).



PROSES REKURSIF

Dengan kata lain, proses (algoritma) untuk memperoleh nilai π dinyatakan sebagai berikut:

1. baca keliling lingkaran, K
2. baca diameter lingkaran, d
3. hitung $\pi = K / d$



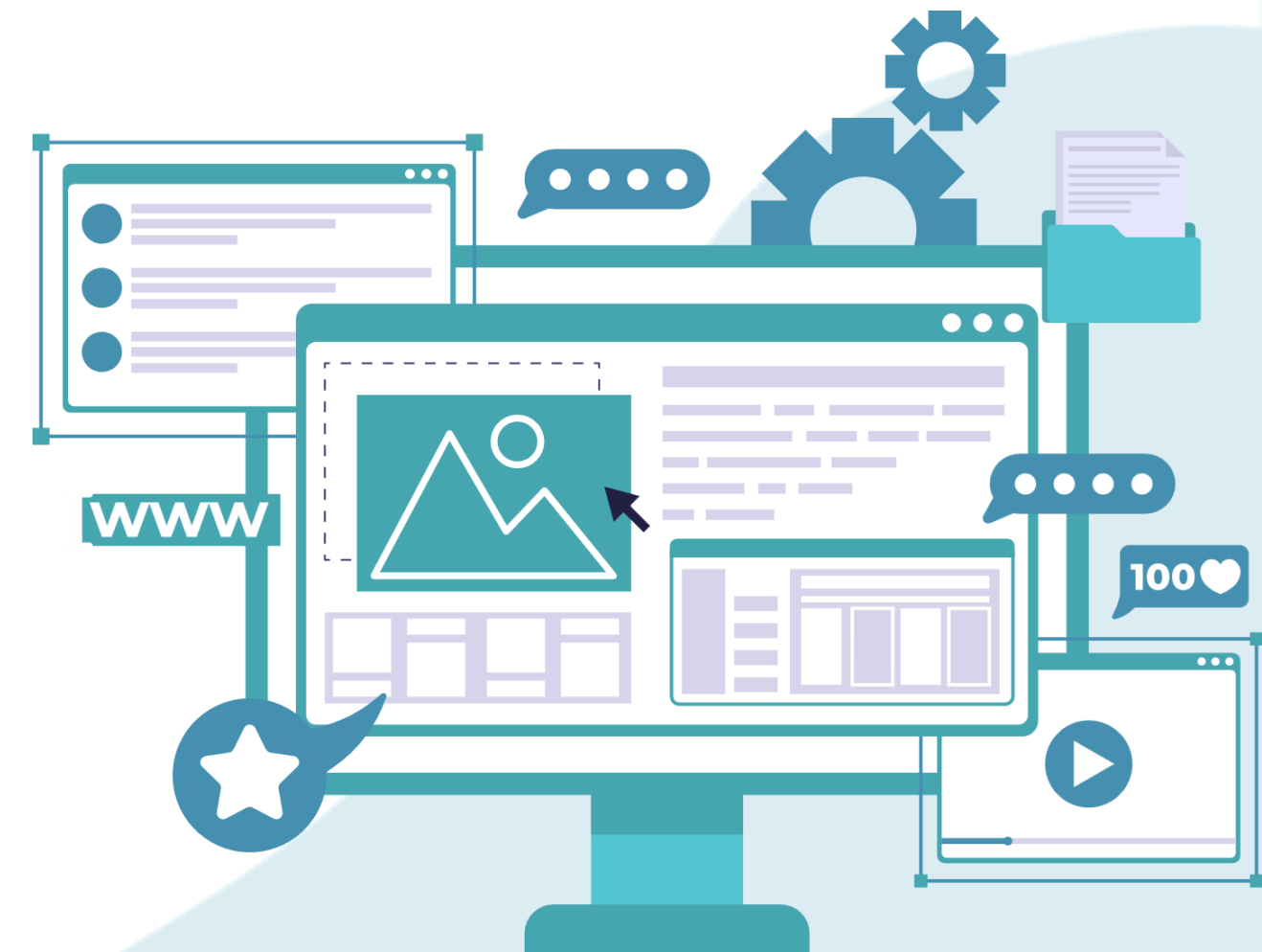
PROSES REKURSIF (2)



Yang jelas, proses tersebut harus berhenti dengan memberikan hasil yang didefinisikan.

Contoh objek lain yang diperoleh dari suatu proses adalah faktorial. Faktorial dari bilangan bulat tak-negatif n didefinisikan sebagai berikut:

$$n! = \begin{cases} 1 & , n = 0 \\ 1 \times 2 \times \dots \times (n-1) \times n & , n > 0 \end{cases}$$



PROSES REKURSIF (3)



Sebagai contoh,

$$0! = 1$$

$$1! = 1$$

$$2! = 1 \times 2$$

$$3! = 1 \times 2 \times 3$$

$$4! = 1 \times 2 \times 3 \times 4$$

Proses untuk menghitung faktorial dari bilangan bulat n tak-negatif

dinyatakan dalam algoritma berikut ini:

PROSES REKURSIF (3)



```
function Faktorial (input n : integer) → integer  
{ mengembalikan nilai n! }
```

DEKLARASI

```
  i : integer  
  F : integer
```

ALGORITMA:

```
  F ← 1  
  I ← 1  
  while i ≤ n do  
    F ← F + i  
    I ← i + 1  
  endwhile  
  { i > n }
```

```
Return F
```


DEFINISI REKURSIF



Definisi rekursif diturunkan secara matematik.
Definisi yang tidak formal menyatakan bahwa sebuah objek dikatakan rekursif jika ia didefinisikan menjadi lebih sederhana dalam terminologi dirinya sendirinya.

DEFINISI REKURSIF



Dalam kehidupan sehari-hari banyak terdapat objek yang rekursif. Tahukah Anda bahwa, kalau diamati dengan saksama, daun pakis (Gambar di samping) dibentuk oleh ranting-ranting daun yang mempunyai pola yang mirip dengan daun pakis itu sendiri. Setiap ranting daun disusun lagi oleh ranting daun dengan pola yang mirip. Objek rekursif yang khas ini disebut fraktal.





CONTOH IMPLEMENTASI PROSES REKURSIF



Untuk memahami implementasi kombinasi perulangan dalam percabangan kita perlu mempelajari penerapannya melalui contoh-contoh implementasi proses rekursif serta memahami cara kerja compiler menangani rekursif.

CONTOH IMPLEMENTASI PROSES REKURSIF

Dengan memahami cara compiler menangani program rekursif dapat membantu kita untuk:

1. Meminimumkan jumlah parameter yang dilewatkan pada waktu pemanggilan,
2. Mengubah algoritma rekursif menjadi program iteratif dengan cara meniru (simulasi) mekanisme pelaksanaan program rekursif.





TERIMA KASIH!